

Veggity: Produce Listing, Freshness & Location Guide

A Step-by-Step Practical Implementation Guide for MERN Stack Beginners

What this guide covers:

You have working Auth (Login & Signup). Your next goal is building the **Produce Listing Engine**.

This blueprint solves your two core requirements in the simplest way possible:

- 1. Perishable Shelf-Life:** Auto-expiring items based on freshness limits (e.g. 1-2 days for spinach, 7 days for potatoes).
- 2. Hyperlocal Discovery:** Using GPS & MongoDB 2dsphere index to show produce only to buyers within 5km - 10km.

1. The Two Core Mechanics Explained (Zero Jargon)

Concept A: The Perishable Shelf-Life Mechanic

When a home grower harvests spinach, it only stays fresh for 1-2 days. If potatoes are harvested, they last 15 days. We solve this without manual intervention:

- When listing, the seller chooses vegetable category & shelf life (e.g., 2 days).
- The backend calculates: `expiresAt = Current Date + (days * 24 hours)`.
- When a buyer browses, the database query checks: `expiresAt: { $gt: new Date() }`. Spoiled items vanish automatically!

Concept B: The 5km / 10km Radius Mechanic (Hyperlocal)

Buyers want fresh veggies from their neighborhood, not from across the country:

- Seller's browser detects GPS coordinates (Latitude & Longitude) using the free HTML5 Geolocation API.
- MongoDB stores coordinates using GeoJSON standard: `[longitude, latitude]`.
- The buyer chooses range (e.g. 5km). MongoDB's built-in `$near` finds all listings within 5,000 meters in milliseconds.

Vegetable Shelf-Life Reference Table:

Category	Examples	Default	Max Allowed
Leafy Greens	Spinach (Palak), Methi, Coriander, Mint	1 Day	2 Days
Soft Veggies	Tomatoes, Cucumber, Green Beans, Okra	2 Days	4 Days
Root / Hardy	Potatoes, Onions, Carrots, Ginger	7 Days	15 Days
Herbs & Fruits	Lemons, Curry Leaves, Chillies, Papaya	3 Days	5 Days

2. Step 1: Database Model (backend/Models/Product.js)

Create this new file in your backend. Note the expiresAt date field and the 2dsphere index:

```
const mongoose = require('mongoose');

const ProductSchema = new mongoose.Schema({
  title: { type: String, required: true },
  category: {
    type: String,
    enum: ['leafy', 'soft-veggies', 'roots', 'herbs'],
    required: true
  },
  price: { type: Number, required: true },
  unit: { type: String, default: 'kg' }, // 'kg', 'grams', 'bunch'
  quantity: { type: Number, required: true },
  harvestDate: { type: Date, default: Date.now },
  shelfLifeDays: { type: Number, required: true },
  expiresAt: { type: Date, required: true },
  locationArea: { type: String, required: true }, // e.g., "Green Park Colony"
  location: {
    type: { type: String, enum: ['Point'], default: 'Point' },
    coordinates: { type: [Number], required: true } // [longitude, latitude]
  },
  seller: { type: mongoose.Schema.Types.ObjectId, ref: 'users', required: true },
  isAvailable: { type: Boolean, default: true },
  createdAt: { type: Date, default: Date.now }
});

// CRITICAL: 2dsphere index enables high-speed geospatial proximity queries
ProductSchema.index({ location: '2dsphere' });

module.exports = mongoose.model('products', ProductSchema);
```

CRITICAL MONGO RULE: Coordinates in MongoDB MUST be ordered **[longitude, latitude]** (Longitude first). GPS APIs like HTML5 Geolocation return (latitude, longitude), so always flip them!

3. Step 2: JWT Auth Middleware (backend/Middlewares/EnsureAuth.js)

Since only logged-in growers should list produce, create this middleware to read the JWT token from the request header:

```
const jwt = require('jsonwebtoken');

const ensureAuthenticated = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  if (!authHeader) {
    return res.status(403).json({ message: 'Unauthorized: Login token missing' });
  }
  try {
    const decoded = jwt.verify(authHeader, process.env.JWT_SECRET);
    req.user = decoded; // Attaches _id and email to req.user
    next();
  } catch (err) {
    return res.status(403).json({ message: 'Token is invalid or expired' });
  }
};

module.exports = ensureAuthenticated;
```

4. Step 3: Product Controller (backend/Controllers/ProductController.js)

Create this controller with two clean methods: adding a listing, and fetching nearby unexpired produce:

Method A: Adding a Listing with Expiry Calculation

```
const ProductModel = require('../Models/Product');

// 1. CREATE PRODUCE LISTING (Grower Side)
const addProduct = async (req, res) => {
  try {
    const { title, category, price, unit, quantity, shelfLifeDays, locationArea, latitude, longitude } = req.body;

    // Category maximum safety boundaries
    const maxLimits = { leafy: 2, 'soft-veggies': 4, roots: 15, herbs: 5 };
    const allowedDays = Math.min(Number(shelfLifeDays), maxLimits[category] || 3);

    // Calculate expiration timestamp
    const expiresAt = new Date(Date.now() + allowedDays * 24 * 60 * 60 * 1000);

    const newProduct = new ProductModel({
      title, category, price, unit, quantity,
      shelfLifeDays: allowedDays,
      expiresAt,
      locationArea,
      location: {
        type: 'Point',
        coordinates: [parseFloat(longitude), parseFloat(latitude)] // [lng, lat]
      },
      seller: req.user._id
    });

    await newProduct.save();
    res.status(201).json({ message: 'Produce listed successfully!', product: newProduct });
  } catch (err) {
    res.status(500).json({ message: 'Failed to create listing', error: err.message });
  }
};
```

Method B: Fetching Fresh Produce within 5km / 10km

```
// 2. GET NEARBY FRESH PRODUCE (Buyer Feed)
const getNearbyProducts = async (req, res) => {
  try {
    const { latitude, longitude, radiusKm = 5 } = req.query;
    if (!latitude || !longitude) {
      return res.status(400).json({ message: 'Latitude and Longitude are required' });
    }

    // Convert kilometers to meters (5km = 5000m)
    const maxDistanceMeters = parseFloat(radiusKm) * 1000;

    const products = await ProductModel.find({
      isAvailable: true,
      expiresAt: { $gt: new Date() }, // ONLY items whose expiry is in the FUTURE!
      location: {
        $near: {
          $geometry: {
            type: 'Point',
            coordinates: [parseFloat(longitude), parseFloat(latitude)]
          },
          $maxDistance: maxDistanceMeters
        }
      }
    }).populate('seller', 'name email');

    res.status(200).json({ count: products.length, products });
  } catch (err) {
    res.status(500).json({ message: 'Failed to fetch nearby produce', error: err.message });
  }
};

module.exports = { addProduct, getNearbyProducts };
```

5. Step 4: Routing & Server Registration

Create backend/Routes/ProductRouter.js:

```
const router = require('express').Router();
const ensureAuthenticated = require('../Middlewares/EnsureAuth');
const { addProduct, getNearbyProducts } = require('../Controllers/ProductController');

// Only logged in growers can post produce
router.post('/add', ensureAuthenticated, addProduct);

// Anyone nearby can view available fresh produce
router.get('/nearby', getNearbyProducts);

module.exports = router;
```

Register in backend/index.js:

```
// Add these lines in backend/index.js
const ProductRouter = require('../Routes/ProductRouter');
// ...
app.use('/products', ProductRouter);
```

6. Step 5: Frontend Seller Form (frontend/src/pages/AddProduce.jsx)

This component auto-detects grower GPS location and provides category-specific shelf-life validation:

```

import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';

function AddProduce() {
  const [form, setForm] = useState({
    title: '', category: 'leafy', price: '', quantity: '',
    unit: 'kg', shelfLifeDays: 1, locationArea: '', latitude: '', longitude: ''
  });
  const [gpsStatus, setGpsStatus] = useState('Detecting location...');
  const navigate = useNavigate();

  // Auto-detect grower's neighborhood coordinates
  useEffect(() => {
    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(
        (pos) => {
          setForm(prev => ({
            ...prev,
            latitude: pos.coords.latitude,
            longitude: pos.coords.longitude
          }));
          setGpsStatus('GPS Location locked!');
        },
        (err) => setGpsStatus('Please allow location access in browser.')
      );
    }
  }, []);

  // Get maximum allowed days for selected category
  const getMaxDays = () => {
    if (form.category === 'leafy') return 2;
    if (form.category === 'soft-veggies') return 4;
    if (form.category === 'roots') return 15;
    return 5;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await fetch('http://localhost:8080/products/add', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': localStorage.getItem('token') // From your login!
      },
      body: JSON.stringify(form)
    });
    const data = await res.json();
    if (res.ok) {
      alert('Vegetable listed successfully!');
      navigate('/home');
    } else {
      alert(data.message || 'Error listing produce');
    }
  };

  // ...render form JSX with inputs and submit button...
}

export default AddProduce;

```

7. Step 6: Frontend Buyer Feed & Freshness Badges

In your **Home.jsx**, add radius selection (5km vs 10km) and calculate remaining fresh hours:

```
// Example function to calculate remaining freshness:
const getFreshnessLabel = (expiresAt) => {
  const diffHours = Math.round((new Date(expiresAt) - new Date()) / (1000 * 60 * 60));
  if (diffHours <= 0) return 'Expired';
  if (diffHours < 24) return `${diffHours} hours fresh left`;
  const days = Math.floor(diffHours / 24);
  return `${days} day${days > 1 ? 's' : ''} freshness remaining`;
};

// Fetching nearby produce using buyer's GPS:
const fetchNearby = async (lat, lng, radiusKm = 5) => {
  const res = await fetch(`http://localhost:8080/products/nearby?latitude=${lat}&longitude=${lng}&radiusKm=${radiusKm}`);
  const data = await res.json();
  setProducts(data.products);
};
```

8. Beginner Troubleshooting & Testing Checklist

Check	What might happen	How to Fix
1. Coordinate Order	Listing appears in another country or 0 results found.	MongoDB requires [longitude, latitude]. Browser gives (lat, lng). Always flip them!
2. 2dsphere Index	MongoDB returns error: <i>planner returned error: unable to find index.</i>	Add <code>ProductSchema.index({ location: '2dsphere' })</code> and restart server.
3. Freshness Filter	Produce doesn't appear on the home feed.	Check if expiresAt is in the past. Ensure system clock & server clock are synchronized.
4. GPS Permission	Browser blocks location prompt.	Click the padlock icon next to URL in Chrome/Edge and set Location to <i>Allow</i> .

You are ready to code!

Follow this order: **Models/Product.js** → **EnsureAuth.js** → **ProductController.js** → **ProductRouter.js** → **AddProduce.jsx** → **Home.jsx**. Take it one file at a time, test with Postman, and check your browser console. You've got this!